

1 неделя

1. Базовые понятия RL

Обучение с подкреплением (Reinforcement Learning, RL) – это направление машинного обучения, в котором агент учится принимать решения путем взаимодействия с окружающей средой. В отличие от обучения с учителем, агент не получает явных правильных ответов: он получает сигнал вознаграждения, который говорит о том, насколько хорошим было его действие, и обучается максимизировать суммарное накопленное вознаграждение.

Ключевые элементы RL:

- Агент (Agent) – объект, который принимает решения и совершает действия в среде. Цель агента – выработать такую стратегию поведения (политику), которая максимизирует долгосрочное вознаграждение.
- Среда (Environment) – все, с чем взаимодействует агент. Среда принимает действие агента, переходит в новое состояние и возвращает вознаграждение.
- Состояние (State, s) – наблюдение агента о текущем положении дел в среде. Может быть полным (если агент видит все) или частичным (если часть информации скрыта).
- Действие (Action, a) – выбор агента в данный момент времени. Множество всех допустимых действий называется пространством действий (action space).
- Вознаграждение (Reward, r) – скалярный сигнал обратной связи, который среда выдает агенту после каждого шага. Может быть положительным (поощрение) или отрицательным (штраф).
- Политика (Policy, π) – стратегия агента: функция, отображающая состояния в действия. Детерминированная политика $\pi(s) = a$ всегда выдает одно действие; стохастическая $\pi(a|s)$ задает распределение вероятностей над действиями.
- Траектория (Trajectory / Episode) – последовательность пар (состояние, действие, вознаграждение). Каждый такой эпизод начинается в начальном состоянии и заканчивается в терминальном.
- Коэффициент дисконтирования (γ , gamma) – число в диапазоне $[0, 1]$, определяющее насколько агент ценит будущие вознаграждения относительно текущих. При $\gamma=0$ агент близорукий (думает только о немедленной награде), при γ близком к 1 – дальновидный.

2. Метод перекрестной энтропии (табличный + нейронная сеть)

Метод перекрестной энтропии (Cross-Entropy Method, CEM) – один из простейших алгоритмов RL, основанный на эволюционном поиске. Идея метода состоит в том, чтобы итерационно улучшать политику, отбирая лучшие эпизоды и обучаясь на них.

Алгоритм (общая схема):

- Сгенерировать N игровых сессий (эпизодов) с текущей политикой.
- Вычислить суммарное вознаграждение каждой сессии.

- Отобрать элитные сессии – те, чье вознаграждение выше р-го перцентиля (например, 70-го).
- Обновить политику, обучив ее на действиях из элитных сессий.
- Повторять до сходимости.

Табличный CEM. В простейшем случае политика представлена таблицей вероятностей. На каждой итерации таблица обновляется как частотная оценка действий из элитных сессий: $\pi[s][a] = \text{count}(s, a \text{ в элитных сессиях}) / \text{count}(s \text{ в элитных сессиях})$. Метод хорошо работает для небольших дискретных пространств состояний, например, в задаче Taxi-v3.

Глубокий CEM (Deep CEM). Когда пространство состояний непрерывно (например, CartPole, MountainCar), таблицу невозможно построить явно. В этом случае политику аппроксимируют нейронной сетью – в данном курсе использовался MLPClassifier из sklearn. Сеть обучается предсказывать вероятности действий по состоянию, а на каждой итерации CEM производится вызов partial_fit на элитных переходах (состояние -> действие).

2 неделя

1. V(s) и Q(s, a) – что это такое?

Функция ценности состояния V(s) (Value Function). $V^\pi(s)$ – ожидаемый суммарный дисконтированный возврат при старте из состояния s и следовании политике π :

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s] = \mathbb{E}_\pi[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots \mid s_t = s]$$

V(s) говорит нам, насколько «выгодно» находиться в состоянии s при данной политике. Оптимальная функция ценности $V^*(s)$ – максимально достижимая ценность состояния при любой политике: $V^*(s) = \max_\pi V^\pi(s)$.

Уравнение Беллмана для V(s):

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s'} P(s' \mid s, a) [R(s, a, s') + \gamma V^\pi(s')]$$

Это рекуррентное соотношение связывает ценность текущего состояния с ценностями следующих. Для оптимальной политики:

$$V^*(s) = \max_a \sum_{s'} P(s' \mid s, a) [R(s, a, s') + \gamma V^*(s')]$$

Функция ценности действия Q(s, a) (Action-Value Function / Q-function). $Q^\pi(s, a)$ – ожидаемый суммарный возврат при старте из состояния s, совершении действия a, и дальнейшем следовании политике π :

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid s_t = s, a_t = a] = \sum_{s'} P(s' \mid s, a) [R(s, a, s') + \gamma V^\pi(s')]$$

Q-функция расширяет V-функцию тем, что явно указывает, какое действие было совершено в состоянии s. Связь между V и Q:

$$V^\pi(s) = \sum_a \pi(a \mid s) Q^\pi(s, a)$$

Оптимальная Q-функция:

$$Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a).$$

Оптимальная политика из Q^* :

$$\pi^*(s) = \arg \max_a Q^*(s, a).$$

Это ключевое преимущество Q-функции над V-функцией: зная Q^* , агент может принимать оптимальные решения без знания модели среды $P(s'|s, a)$. Именно поэтому алгоритмы, основанные на Q-функции (Q-learning, DQN), называются model-free.

2. Метод итерации по политике (Policy Iteration)

Policy Iteration (PI) – это динамическое программирование для нахождения оптимальной политики в MDP с известной моделью среды. Метод чередует два шага: оценку текущей политики и ее улучшение.

Шаг 1 – Оценка политики (Policy Evaluation). Для фиксированной политики π вычисляется точная функция ценности V^{π} путем решения системы линейных уравнений Беллмана.

Шаг 2 – Улучшение политики (Policy Improvement). Используя вычисленные значения V^{π} , обновляем политику жадно.

Сходимость. Алгоритм завершается, когда политика перестает меняться, то есть $\pi_{\text{new}} = \pi$. В этот момент достигнута оптимальная политика π^* .

3 неделя

1. Q-learning, SARSA

Q-learning и SARSA – это алгоритмы обучения с подкреплением на основе временных разностей (Temporal Difference, TD), которые обучаются прямо в процессе взаимодействия со средой без знания ее модели (model-free). Оба алгоритма поддерживают таблицу Q-значений и обновляют ее после каждого перехода.

Q-learning является off-policy алгоритмом: для обновления Q используется значение оптимального действия в s' ($\max Q$), независимо от того, какое действие реально будет выбрано политикой. Это означает, что Q-learning напрямую аппроксимирует оптимальную Q^* , даже если агент исследует среду с помощью epsilon-жадной политики.

SARSA (on-policy TD). Название SARSA происходит от кортежа (State, Action, Reward, State, Action). Обновление:

Здесь a' – действие, которое агент реально выберет в s' согласно своей текущей политике. SARSA является on-policy алгоритмом: он обучает ту же политику, по которой действует. Это делает SARSA более консервативным: агент учитывает возможность выбора случайного действия при ϵ -жадной политике.

Различие поведения в CliffWalking. В задаче со скалой (CliffWalking) Q-learning находит оптимальный путь вдоль края скалы (короткий, но рискованный), так

как игнорирует случайные действия. SARSA учитывает, что иногда будет выбрано случайное действие, и поэтому предпочитает более безопасный путь подальше от скалы. Это наглядно демонстрирует разницу между on-policy и off-policy методами.

2. Exploration vs Exploitation

Одна из центральных проблем RL – баланс между исследованием (exploration) новых действий для получения информации о среде и использованием (exploitation) уже известных хороших действий для максимизации вознаграждения. Если агент всегда эксплуатирует текущие знания, он может застрять в локальном оптимуме; если всегда исследует – никогда не наберет достаточно вознаграждения.

Основные стратегии исследования:

- ϵ -жадная политика (Epsilon-Greedy).
- Softmax исследование.
- UCB (Upper Confidence Bound).
- Curiosity-driven exploration / Intrinsic motivation.
- Thompson sampling.

Практические рекомендации: начинать с высоким ϵ (1.0) и постепенно уменьшать. Скорость уменьшения зависит от сложности задачи: слишком быстрое снижение ϵ ведет к преждевременной сходимости к субоптимальной политике; слишком медленное – к неэффективному обучению на поздних этапах.

4 неделя

1. DQN (Deep Q-Network)

DQN – алгоритм, который объединяет Q-learning с глубокими нейронными сетями для решения задач с высокоразмерными пространствами состояний (например, пиксели экрана Atari).

Основная идея. Функция $Q(s,a;\theta)$ аппроксимируется нейронной сетью с параметрами θ . Сеть принимает на вход состояние s и выдает Q-значения для всех действий одновременно.

2. Experience Replay (Буфер воспроизведения опыта)

При прямом обучении на последовательных переходах (s_t, a_t, r_t, s_{t+1}) обучающие примеры сильно скоррелированы во времени: соседние переходы порождены одним и тем же состоянием среды. Это нарушает предположение о независимости и одинаковой распределенности обучающих данных, на котором основан стохастический градиентный спуск. Кроме того, агент мог бы «забыть» редкие, но важные переходы.

Решение: буфер воспроизведения (Replay Buffer). Все переходы $(s, a, r, s', done)$ сохраняются в кольцевой буфер фиксированного размера (обычно 10^5 – 10^6). На

каждом шаге обучения из буфера случайно сэмплируется мини-батч переходов. Это решает сразу несколько проблем:

- Декорреляция данных: случайная выборка разрывает временные корреляции.
- Повторное использование данных: один и тот же переход может использоваться для обновления параметров несколько раз, повышая эффективность использования данных.
- Стабилизация распределения: буфер содержит переходы от разных версий политики, что смягчает нестационарность обучающего распределения.

Реализация: при заполнении буфера сверх максимального размера самые старые переходы удаляются (FIFO). Сэмплирование производится равномерно (в базовом варианте) или с приоритетами (Prioritized Experience Replay).

3. Target Network (Целевая сеть)

Проблема без целевой сети. В базовом Q-learning с нейронной сетью целевое значение вычисляется как $r + \gamma \cdot \max_{a'} Q(s', a'; \theta)$. Однако параметры θ обновляются на каждом шаге, что означает: мы гонимся за движущейся целью (moving target). Это вызывает нестабильность и расходимость обучения – параметры сети влияют одновременно на предсказания и на цели.

Решение: целевая сеть $Q(s, a; \theta^-)$. Вводится отдельная копия сети с параметрами θ^- , которая используется только для вычисления целевых Q-значений: $\text{target} = r + \gamma \cdot \max_{a'} Q(s', a'; \theta^-)$. Параметры θ^- обновляются редко – раз в N шагов (например, каждые 10 000 шагов) путем прямого копирования $\theta^- \leftarrow \theta$. В промежутках θ^- заморожены, что делает цель стабильной.

Интуиция: каждый цикл обновления θ^- – это один шаг γ -сжатия в аппроксимации оптимальных Q-значений. Слишком частые обновления $\theta^- \rightarrow$ нестабильность; слишком редкие $\rightarrow$ медленное обучение. Оптимальное значение N определяется экспериментально (в Nature DQN – 10 000 шагов).

4. Double DQN

Проблема переоценки (overestimation bias) в DQN. Стандартный DQN использует $\max_{a'} Q(s', a'; \theta^-)$ для выбора и оценки действия одновременно. Из-за шума в Q-значениях операция $\max$ систематически выбирает переоцененные действия, что приводит к завышенной оценке $V(s')$ и нестабильному обучению.

Решение: Double DQN. Разделяем выбор действия и его оценку на два независимых шага:

- Выбор действия производится основной сетью.
- Оценка Q-значения производится целевой сетью.

Итоговая цель: $\text{target} = r + \gamma \cdot Q(s', \arg\max_{a'} Q(s', a'; \theta); \theta^-)$. Так как выбор и оценка производятся разными сетями с разными параметрами, систематическое завышение устраняется. Double DQN показывает более стабильное и точное обучение на большинстве игр Atari при тех же вычислительных затратах, что и стандартный DQN.

6 неделя

1. Policy Gradient (Градиент политики)

Policy Gradient – класс алгоритмов RL, которые напрямую оптимизируют параметры политики θ , не строя явной функции ценности. Политика $\pi_\theta(a|s)$ параметризуется нейронной сетью, и мы оптимизируем ожидаемый суммарный возврат $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)]$ методом градиентного подъема.

Теорема о градиенте политики (Policy Gradient Theorem):

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot G_t \right]$$

Интуиция: если в состоянии s агент выбрал действие a и получил хороший возврат G_t , то увеличиваем вероятность $\pi_\theta(a|s)$. Если возврат плохой – уменьшаем.

Трюк log-вероятности. Так как $\nabla_\theta \pi_\theta = \pi_\theta \cdot \nabla_\theta \log \pi_\theta$, градиент можно оценить как стохастический градиент следующей суррогатной функции потерь:

$$L(\theta) = \frac{1}{N} \sum_{s_i, a_i} \log \pi_\theta(a_i | s_i) \cdot G_t$$
 . Отрицательное значение этой функции минимизируется, что эквивалентно максимизации $J(\theta)$.

Baseline для снижения дисперсии. Формула градиента политики остается несмещенной, если вычесть из G_t любую базовую функцию $b(s)$, не

зависящую от действия:

$$\nabla_\theta J(\theta) = \mathbb{E} \left[\sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot (G_t - b(s_t)) \right]$$

2. REINFORCE

REINFORCE – простейший алгоритм policy gradient на основе Monte Carlo оценки возвратов. Является частным случаем теоремы о градиенте политики, где G_t вычисляется по полной траектории эпизода.

Алгоритм:

- Сгенерировать полный эпизод с текущей политикой π_θ .
- Вычислить дисконтированные возвраты G_t для каждого t .
- Вычислить суррогатную функцию: $L(\theta)$.
- Обновить параметры.

Реализация в PyTorch. Политика представлена нейронной сетью, выдающей логиты. Вероятности: $\text{probs} = \text{softmax}(\text{logits})$. Log-вероятности: $\text{log_probs} = \text{log_softmax}(\text{logits})$. Log-вероятности выбранных действий: $\text{log_probs_for_actions} = \sum_a \text{log_probs} \cdot \text{one_hot}(\text{actions})$. Потеря: $\text{loss} = -\text{mean}(\text{log_probs_for_actions} \cdot G_t)$.

Недостатки REINFORCE:

- Высокая дисперсия: G_t является Monte Carlo оценкой, зависящей от всей будущей траектории.
- Неэффективность данных: каждая траектория используется для обновления только один раз (on-policy).

- Slow convergence: требует много эпизодов для стабильного обучения.

3. Actor-Critic

Actor-Critic – это архитектура RL, которая объединяет преимущества policy gradient (Actor) и методов обучения функции ценности (Critic), устраняя главный недостаток REINFORCE – высокую дисперсию.

Компоненты:

- Actor – политика $\pi_{\theta}(a|s)$, параметризованная нейронной сетью. Отвечает за выбор действий и оптимизируется методом policy gradient.
- Critic – функция ценности $V_{\phi}(s)$ (или $Q_{\phi}(s,a)$), параметризованная другой сетью. Оценивает, насколько хороши состояния, и используется для вычисления advantage.

Обновления:

Critic обновляется минимизацией TD-ошибки.

Actor обновляется policy gradient с advantage в качестве веса.

Интуиция advantage: если $A(s,a) > 0$, действие оказалось лучше ожидаемого – увеличиваем его вероятность. Если $A(s,a) < 0$ – действие хуже ожидаемого – уменьшаем. Критик предоставляет «ожидаемый уровень» для сравнения, снижая дисперсию.

Варианты архитектуры:

- A2C (Advantage Actor-Critic) – синхронная версия с несколькими параллельными агентами.
- A3C (Asynchronous Advantage Actor-Critic) – асинхронная версия, где несколько агентов параллельно обновляют глобальную сеть.
- PPO (Proximal Policy Optimization) – ограничивает размер шага обновления политики для стабильности.
- SAC (Soft Actor-Critic) – максимизирует энтропию политики вместе с возвратом.

7 неделя

1. Применение RNN для генерации последовательностей

Рекуррентные нейронные сети (RNN) – класс архитектур, предназначенных для работы с последовательными данными переменной длины. В отличие от обычных нейронных сетей, RNN сохраняют скрытое состояние h_t , которое передается от шага к шагу и накапливает контекст предыдущих элементов последовательности.

Принцип работы. На каждом шаге t RNN получает входной токе и предыдущее скрытое состояние, и вычисляет новое скрытое состояние и выход. Проблема базовой RNN – затухание градиентов (vanishing gradient) при длинных последовательностях: сигнал ошибки ослабевает при обратном распространении через множество шагов.

LSTM (Long Short-Term Memory). Решает проблему затухания градиентов через механизм вентилях (gates):

- Входной вентиль контролирует, какая информация записывается в память.
- Вентиль забывания контролирует, какая часть предыдущей памяти стирается.
- Выходной вентиль контролирует, какая часть памяти передается.
- Ячейка памяти C_t хранит долгосрочную информацию.

Архитектура Encoder-Decoder (seq2seq). Для задач перевода последовательностей (machine translation, g2p) используется архитектура энкодер-декодер:

- Энкодер читает входную последовательность (например, ивритское слово) символ за символом и создает вектор контекста – финальное скрытое состояние, сжимающее всю информацию об входе.
- Декодер принимает вектор контекста как начальное состояние и генерирует выходную последовательность символ за символом, используя предыдущий выход как вход на следующем шаге.
- Механизм внимания (Attention) улучшает архитектуру, позволяя декодеру на каждом шаге «заглядывать» в разные позиции кодированной последовательности вместо использования только финального состояния энкодера.

Генерация последовательностей. Модель seq2seq обучается методом максимального правдоподобия: на входе декодера используются истинные предыдущие токены, а не предсказанные. Функция потерь – кроссэнтропия по всем позициям. При генерации (инференсе) применяются:

- Жадная стратегия (Greedy decoding): на каждом шаге выбирается токен с максимальной вероятностью. Быстро, но не оптимально.
- Beam search: поддерживает топ-k гипотез на каждом шаге, что улучшает качество перевода.
- Сэмплирование (Sampling): выбор токена случайно согласно предсказанному распределению – для разнообразия генерации.

2. Применение RL к дообучению моделей генерации последовательностей

Self-Critical Sequence Training (SCST). Алгоритм SCST применяет метод policy gradient REINFORCE с умным baseline. Идея: агент (seq2seq модель) генерирует два перевода для одного входа – один путем сэмплирования (sampling) и один жадно (greedy). Reward = отрицательное расстояние Левенштейна (или другая дифференцируемая метрика). Baseline = reward жадного перевода.

Градиент SCST:

$$\nabla_{\theta} J \approx \frac{1}{T} \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \cdot (R(x, y_{\text{sample}}) - R(x, y_{\text{greedy}}))$$

$A = R(x, y_{\text{sample}}) - R(x, y_{\text{greedy}})$ показывает, насколько случайный перевод оказался лучше или хуже жадного. Если лучше – увеличиваем вероятности выбранных токенов; если хуже – уменьшаем.

Преимущества SCST:

- Напрямую оптимизирует целевую метрику (Edit Distance, BLEU), а не суррогатную.
- Baseline вычисляется без дополнительной параметризованной сети – только жадный перевод текущей модели.
- Низкая дисперсия по сравнению с классическим REINFORCE (baseline хорошо коррелирует с reward).
- Позволяет обучаться на нескольких правильных переводах одновременно (в word_to_translation словаре несколько правильных ответов).

Практические аспекты применения RL к seq2seq:

- Предобучение на supervised данных обязательно: начинать RL с нуля крайне нестабильно, так как начальная политика слишком хаотична.
- Низкая скорость обучения ($lr=1e-5$) против supervised ($lr=1e-3$): градиенты policy gradient шумнее.
- Энтропийная регуляризация предотвращает коллапс политики к детерминированной (когда модель перестает исследовать).
- UREX (Under-appreciated Reward Exploration) – продвинутая техника для поиска недооцененных действий.
- Оценка через Edit Distance (Levenshtein): не дифференцируема, но может использоваться как reward в RL. Позволяет учитывать все допустимые переводы слова.